LAPORAN PRAKTIKUM
PENGUJIAN PERANGKAT LUNAK
PERTEMUAN 2
UNIT TESTING LIFECYCLE

Disusun oleh:

Nama	: Daffa' Abiyyu Dzulfiqar
NIM	: 24/539118/SV/24566
Kelas	: A1
Dosen Pengampu	: Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2025

Daftar Isi

A. Tujuan Praktikum **2**

B. Hasil dan Pembahasan **2**

1. Implementasi kelas `Wallet` 2
2. Implementasi Test Lifecycle pada `WalletTest` 4
3. Implementasi anotasi `@Order` 5
4. Hasil Eksekusi Test Lifecycle 5

C. Kesimpulan **9**

A. Tujuan Praktikum

1. Mahasiswa mampu menjelaskan tahapan eksekusi pengujian lifecycle menggunakan kerangka kerja JUnit pada bahasa Java.
2. Mahasiswa dapat mengidentifikasi fungsi spesifik dari berbagai anotasi dasar lifecycle yang disediakan oleh JUnit.
3. Mahasiswa mampu mengimplementasikan anotasi JUnit dengan tepat pada kode pengujian unit perangkat lunak.
4. Mahasiswa dapat merancang skenario pengujian unit yang terstruktur berdasarkan pemahaman terkait anotasi dan lifecycle JUnit.

B. Hasil dan Pembahasan

1. Implementasi kelas `Wallet`

Kelas `Wallet` dirancang sebagai representasi sederhana dari dompet digital yang memiliki fungsi-fungsi dasar pengelolaan keuangan. Kelas ini disimpan dalam paket `src/main/java`. Terdapat tiga atribut utama yang mendefinisikan keadaan (*state*) dari sebuah objek `Wallet`:

- `owner`: Variabel bertipe `String` yang menyimpan nama pemilik dompet.
- `cash`: Variabel bertipe `double` untuk menyimpan jumlah uang tunai yang tersedia. Tipe data `double` dipilih untuk mengakomodasi nilai pecahan desimal pada mata uang.
- `cards`: Berupa `List<String>` yang menyimpan daftar kartu (misalnya kartu debit atau kredit) yang ada di dalam dompet. Inisialisasi daftar ini dilakukan di dalam konstruktor menggunakan `ArrayList`.

Penjelasan implementasi atribut dan konstruktor dapat dilihat pada Kode 1. Konstruktor kelas menerima dua parameter awal yaitu nama pemilik dan saldo awal, kemudian menginisialisasi daftar kartu sebagai list kosong yang dapat diisi.

Listing 1: Konstruktor dan atribut kelas Wallet

```java
public class Wallet {
    private String owner;
    private final List<String> cards;
    private double cash;

    public Wallet(String owner, double initialCash) {
        this.owner = owner;
        this.cash = initialCash;
        this.cards = new ArrayList<>();
    }
}
```

Fungsi utama dompet diimplementasikan melalui beberapa metode. Metode `deposit` (Kode 2) berfungsi untuk menambah saldo uang tunai ke dalam dompet. Di dalam metode ini terdapat logika validasi sederhana, saldo hanya akan bertambah jika nominal yang dimasukkan (`amount`) bernilai positif. Jika nilai nol atau negatif, operasi diabaikan untuk menjaga kesamaan data.

Listing 2: Implementasi metode deposit

```java
public void deposit(double amount) {
    if (amount > 0) {
        this.cash += amount;
    }
}
```

Selanjutnya, metode `withdraw` (Kode 3) untuk menangani penarikan uang. Metode ini memiliki nilai *return* bertipe `boolean` untuk menandakan keberhasilan/kegagalan transaksi. Jika jumlah penarikan melebihi saldo saat ini, metode akan langsung mengembalikan nilai `false` tanpa mengurangi saldo. Sebaliknya, apabila saldo mencukupi, saldo akan dikurangi dan metode mengembalikan `true`.

Listing 3: Implementasi metode withdraw

```java
public boolean withdraw(double amount) {
    if (amount > this.cash) {
        return false;
    }
    this.cash -= amount;
    return true;
}
```

Selain pengelolaan uang, kelas ini juga menangani penyimpanan kartu melalui metode `addCard` dan `removeCard` (Kode 4). Metode `addCard` menerima nama bank dan nomor rekening, lalu menggabungkannya menjadi satu string berformat sebelum menambahkannya ke dalam list. Metode `removeCard` menghapus kartu yang terakhir kali ditambahkan (prinsip *Last-In-First-Out* untuk tumpukan kartu), serta mengembalikan nilai kartu yang dihapus tersebut. Jika dompet tidak memiliki kartu, metode mengembalikan `null`.

Listing 4: Implementasi metode menambah dan hapus kartu

```java
public void addCard(String bankName, String
    accountNumber) {
    String cardInfo = bankName + " (" + accountNumber +
        ")";
    this.cards.add(cardInfo);
}

public String removeCard() {
    if (!cards.isEmpty()) {
        return cards.removeLast();
    }
    return null;
}
```

Selain metode-metode tersebut, terdapat juga metode utilitas tambahan seperti `hasCard` untuk mengecek keberadaan kartu tertentu, dan `hasEnoughFunds` untuk memvalidasi kecukupan saldo. Kelas ini juga dilengkapi dengan metode *getter* dan *setter* untuk setiap variabel privat, serta *override* metode `toString()` untuk merepresentasikan objek `Wallet` dalam bentuk string yang informatif.

2. Implementasi Test Lifecycle pada `WalletTest`

Pengujian unit pada kelas `Wallet` dilakukan di dalam kelas `WalletTest`. Pada kelas ini, diimplementasikan beberapa anotasi lifecycle dari JUnit 5 untuk mengatur tahapan persiapan (*setup*) dan pembersihan (*teardown*) sebelum dan sesudah pengujian dijalankan. Anotasi yang digunakan meliputi `@BeforeAll`, `@AfterAll`, `@BeforeEach`, dan `@AfterEach`.

Listing 5: Implementasi anotasi lifecycle pada WalletTest

```java
    @BeforeAll
    static void initClass() {
        System.out.println("WalletTest Started");
    }

    @AfterAll
    static void cleanClass() {
        System.out.println("WalletTest Finished");
    }

    @BeforeEach
    void initMethod() {
        wallet = new Wallet("Daffa", 50000.0);
        System.out.println("[Setup] " + wallet);
    }

    @AfterEach
    void cleanMethod() {
        wallet = null;
    }
```

Berdasarkan Kode 5, berikut adalah penjelasan fungsi masing-masing anotasi:

- `@BeforeAll`: Menandakan bahwa metode `initClass()` akan dieksekusi tepat satu kali sebelum seluruh metode pengujian di dalam kelas dijalankan. Metode ini harus bersifat `static`. Pada kasus ini, metode digunakan untuk mencetak penanda bahwa pengujian kelas dimulai.
- `@AfterAll`: Menandakan bahwa metode `cleanClass()` akan dieksekusi tepat satu kali setelah seluruh metode pengujian selesai dijalankan. Sama seperti `@BeforeAll`, metode ini juga harus bersifat `static`.
- `@BeforeEach`: Metode `initMethod()` akan dieksekusi sebelum setiap metode `@Test` dijalankan. Hal ini memastikan bahwa setiap skenario pengujian mendapatkan objek `Wallet` yang baru dan independen dengan saldo awal 50000.0, sehingga status dari satu pengujian tidak memengaruhi pengujian lainnya.
- `@AfterEach`: Metode `cleanMethod()` akan dieksekusi setelah setiap metode `@Test` selesai. Pada kode ini, objek `wallet` diatur menjadi `null` untuk membersihkan memori atau *state* setelah pengujian.

3. Implementasi anotasi `@Order`

Secara bawaan, JUnit tidak menjamin urutan eksekusi metode pengujian. Namun, pada beberapa kasus, kita mungkin ingin mengontrol urutan eksekusi tersebut. JUnit 5 menyediakan anotasi `@Order` yang bekerja sama dengan anotasi tingkat kelas `@TestMethodOrder`.

Listing 6: Implementasi anotasi @Order pada WalletTest

```java
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class WalletTest {
    // ... kode lifecycle ...

    @Test
    @Order(1)
    void testConstructorAndFields() {
        // ... kode pengujian ...
    }

    @Test
    @Order(2)
    void testDepositMoney() {
        // ... kode pengujian ...
    }

    // ... pengujian lainnya dengan @Order(3), @Order(4), @Order(5) ...
}
```

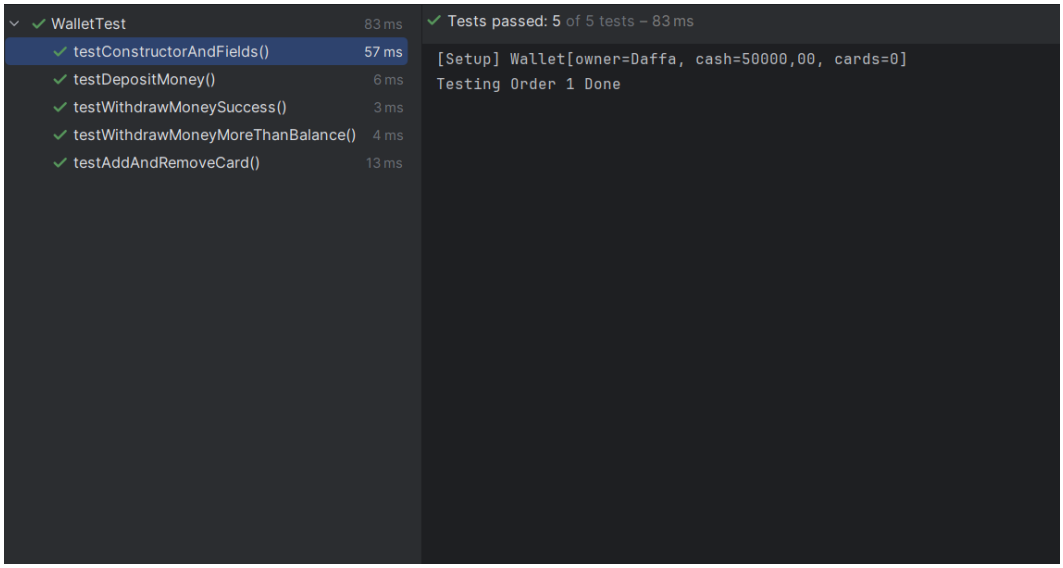
Pada Kode 6, kelas `WalletTest` ditambahkan anotasi `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)`. Anotasi ini menginstruksikan JUnit untuk mengeksekusi metode pengujian berdasarkan urutan yang ditentukan oleh anotasi `@Order` pada masing-masing metode.

Setiap metode pengujian (`@Test`) kemudian diberikan anotasi `@Order(n)`, di mana `n` adalah angka bulat yang menentukan prioritas eksekusi. Metode dengan nilai `n` yang lebih kecil akan dieksekusi lebih awal. Pada `WalletTest`, pengujian diurutkan mulai dari `@Order(1)` untuk menguji konstruktor, hingga `@Order(5)` untuk menguji penambahan dan penghapusan kartu. Penggunaan `@Order` ini sangat berguna untuk menstrukturkan alur laporan pengujian agar lebih mudah dibaca dan dipahami secara berurutan.

4. Hasil Eksekusi Test Lifecycle

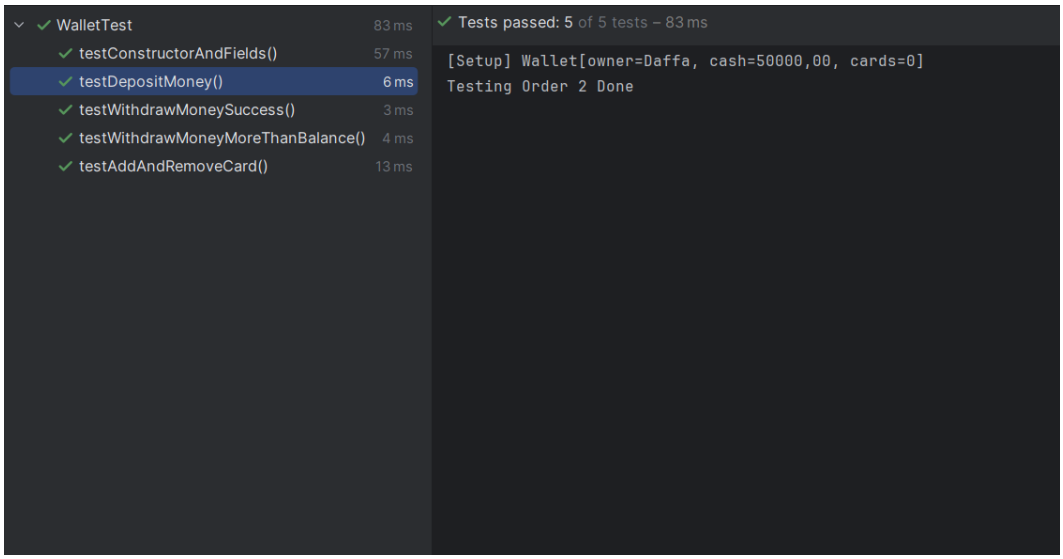
Setelah seluruh kode pengujian dan anotasi lifecycle diimplementasikan, tahap selanjutnya adalah mengeksekusi kelas `WalletTest`. Saat dijalankan, JUnit akan secara otomatis mendeteksi dan mengeksekusi metode-metode yang memiliki anotasi `@Test` sesuai dengan urutan yang telah didefinisikan oleh `@Order`. Berikut adalah hasil eksekusi dari kelima skenario pengujian yang telah dirancang:

1. Verifikasi konstruktor dan atribut awal.



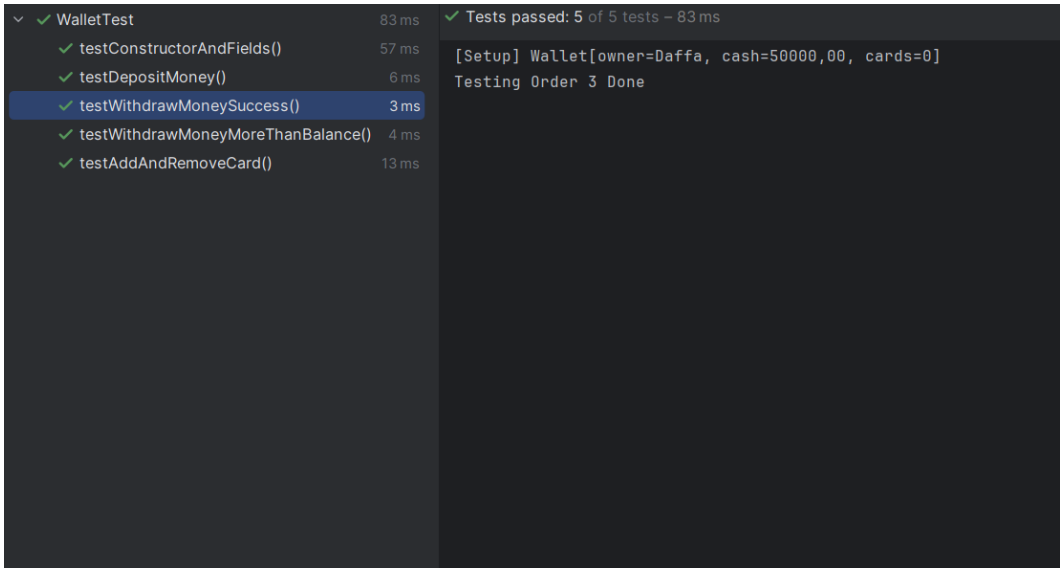
Gambar 1: Hasil verifikasi konstruktor dan atribut awal.

2. Verifikasi penambahan saldo (deposit).



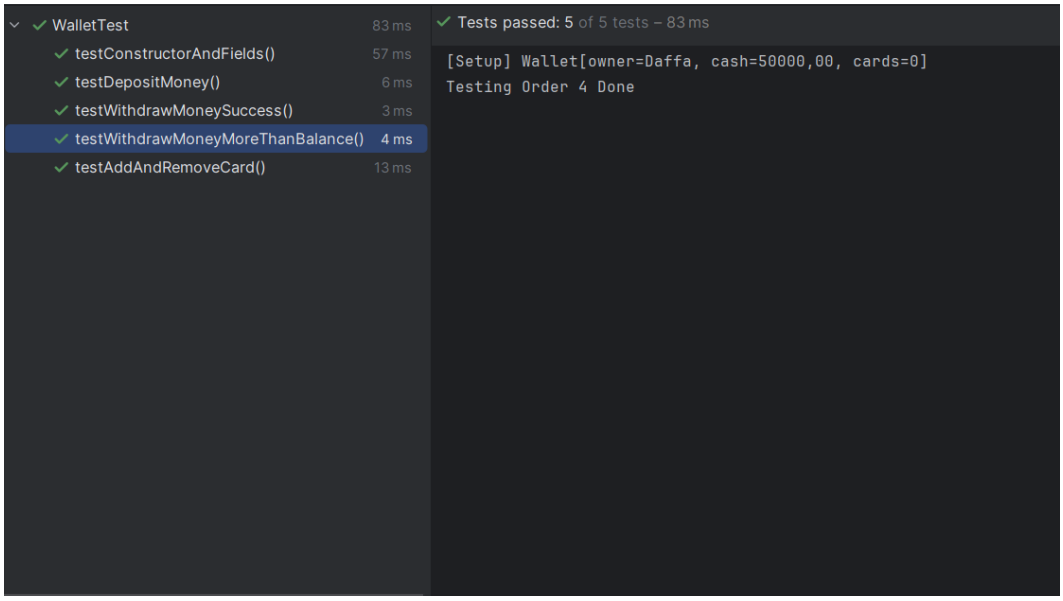
Gambar 2: Hasil verifikasi penambahan saldo (deposit).

3. Verifikasi penarikan uang yang valid.



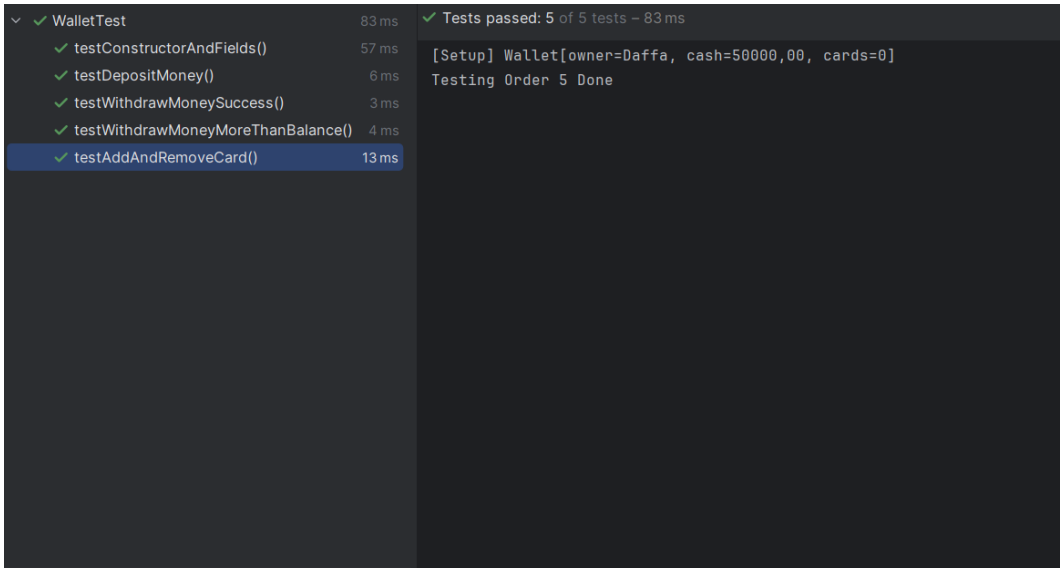
Gambar 3: Hasil verifikasi penarikan uang yang valid.

4. Verifikasi penolakan penarikan uang saat saldo kurang.



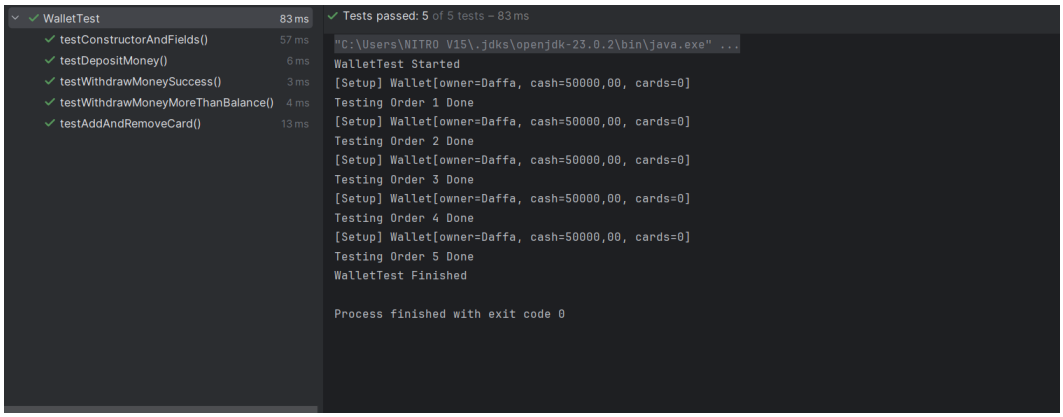
Gambar 4: Hasil verifikasi penolakan penarikan uang saat saldo kurang

5. Verifikasi pengelolaan kartu (tambah/hapus).



Gambar 5: Hasil verifikasi pengelolaan kartu (tambah/hapus)

6. Pengujian lifecycle pada `WalletTest`



Gambar 6: Hasil pengujian lifecycle

Berdasarkan hasil eksekusi pada gambar-gambar di atas, terlihat bahwa seluruh skenario pengujian (5 dari 5 tes) berhasil dilewati (*passed*) tanpa adanya kegagalan. Output pada konsol juga menunjukkan bahwa anotasi lifecycle seperti `@BeforeAll`, `@BeforeEach`, dan `@AfterAll` berjalan sesuai dengan urutan yang diharapkan, mencetak pesan inisialisasi dan pembersihan dengan benar. Selain itu, eksekusi metode pengujian juga terurut dengan tepat dari `Order 1` hingga `Order 5`, membuktikan bahwa anotasi `@Order` berfungsi dengan baik dalam mengatur alur pengujian.

C. Kesimpulan

Berdasarkan praktikum ini, dapat disimpulkan bahwa pemahaman dan penerapan *test lifecycle* sangat krusial dalam membangun skenario pengujian unit yang andal dan terstruktur. Melalui implementasi kelas `WalletTest`, mahasiswa telah berhasil mengaplikasikan berbagai anotasi lifecycle seperti `@BeforeAll`, `@AfterAll`, `@BeforeEach`, dan `@AfterEach`. Anotasi-anotasi ini terbukti efektif dalam mengelola tahapan persiapan (*setup*) dan pembersihan (*teardown*) lingkungan pengujian secara otomatis. Dengan demikian, setiap metode uji dapat berjalan secara independen tanpa terpengaruh oleh *state* atau sisa data dari pengujian sebelumnya, yang pada akhirnya memastikan bahwa setiap skenario diuji dalam kondisi yang konsisten dan terkontrol.

Selain itu, dapat disimpulkan pentingnya mengatur urutan eksekusi pengujian menggunakan anotasi `@Order` dan `@TestMethodOrder` sebagai bagian dari manajemen lifecycle. Meskipun secara prinsip pengujian unit idealnya bersifat independen, kemampuan untuk mengontrol urutan eksekusi sangat bermanfaat dalam menstrukturkan alur laporan dan mempermudah proses *debugging*. Hasil eksekusi yang menunjukkan tingkat keberhasilan 100% (5 dari 5 tes *passed*) membuktikan bahwa logika bisnis pada kelas `Wallet`, termasuk metode-metode utilitas tambahan seperti `hasCard` dan `hasEnoughFunds`, telah diimplementasikan dengan benar. Secara keseluruhan, penguasaan terhadap lifecycle pengujian ini menjadi fondasi yang kuat bagi mahasiswa dalam mengembangkan perangkat lunak yang berkualitas dan minim *bug*.